Modelos matemáticos del reconocimiento de audio

Andrés Felipe Verbel Pacheco — Desarrollo Web Full Stack, CUC

Documento de referencia: todas las fórmulas que usa la demo, con los valores reales de sus parámetros y el lugar del código donde se implementan.

---

1. Muestreo y representación digital
2. Transformada discreta de Fourier (DFT)
3. Transformada rápida de Fourier (FFT)
4. Ventana de Hann
5. Transformada de tiempo corto y espectrograma
6. Detección de picos (constelación)
7. Pares de picos y hash de 24 bits
8. Índice invertido y votación
9. Ruido y relación señal-ruido
10. Colisiones de hash
11. Síntesis de las canciones de prueba
12. Visualización 2D y 3D
13. Métricas de evaluación
14. Tabla de parámetros y mapa del código

1. Muestreo y representación digital

El audio continuo $x_a(t)$ se convierte en una secuencia de muestras tomadas cada $1/f_s$ segundos. Para que cada frecuencia quede representada sin ambigüedad (sin «aliasing») solo pueden existir componentes por debajo de la frecuencia de Nyquist:

$$x[n] = x_a\!\left(\tfrac{n}{f_s}\right), \qquad f_{\text{Nyq}} = \tfrac{f_s}{2} = 4.000 \text{ Hz} \quad (f_s = 8.000 \text{ Hz}). \tag{1}$$

Cualquier audio que entra a la demo (archivo, micrófono) se remuestrea a $f_s = 8.000$ Hz con el remuestreador del navegador (`OfflineAudioContext`), y los canales estéreo se mezclan a mono. Las canciones de la base se guardan como WAV PCM de 16 bits; la conversión entre entero s y amplitud real es:

$$x = \frac{s}{32767}, \qquad s = \operatorname{round}(32767\,x), \quad s \in [-32768,\, 32767]. \tag{2}$$

Código: `lib/sintesis.js` → `escribirWav`, `leerWav`; `public/app.js` → `a8k`, `decodificar`.

2. Transformada discreta de Fourier (DFT)

Para un bloque de N muestras, la DFT descompone la señal en N componentes sinusoidales complejas; $|X[k]|$ es su amplitud y $\arg X[k]$ su fase:

$$X[k] = \sum_{n=0}^{N-1} x[n]\, e^{-j2\pi kn/N}, \qquad k = 0, 1, \ldots, N-1. \tag{3}$$

El índice k (bin) corresponde a una frecuencia concreta, y la separación entre bins es la **resolución en frecuencia**:

$$f_k = \frac{k\, f_s}{N}, \qquad \Delta f = \frac{f_s}{N} = 7,8125 \text{ Hz}, \qquad k(f) = \frac{f N}{f_s}. \tag{4}$$

Ejemplo: la nota La4 (440 Hz) cae en $k = 56,32$, entre los bins 56 y 57. Como la señal de audio es real, el espectro es simétrico, así que solo hacen falta los primeros $N/2$ bins, y se cumple la conservación de energía (teorema de Parseval):

$$X[N-k] = \overline{X[k]}, \qquad \sum_{n=0}^{N-1} |x[n]|^2 = \frac{1}{N} \sum_{k=0}^{N-1} |X[k]|^2. \tag{5}$$

Calculada directamente, la DFT necesita N^2 multiplicaciones complejas (1.048.576 para $N = 1024$).

Código: `lib/huella.js` → `dft` (referencia para validar la FFT). Verificación: pruebas «FFT coincide con la DFT» y «Parseval» en `test/huella.test.js`.

3. Transformada rápida de Fourier (FFT)

El algoritmo de Cooley–Tukey calcula exactamente la misma DFT con menos operaciones. Separa las muestras pares e impares: si $E[k]$ y $O[k]$ son las DFT de tamaño $N/2$ de cada grupo y $W_N = e^{-j2\pi/N}$ (factor de giro), entonces

$$\begin{aligned} X[k] &= E[k] + W_N^k\, O[k], \\ X[k+\tfrac{N}{2}] &= E[k] - W_N^k\, O[k], \end{aligned} \qquad 0 \le k < \tfrac{N}{2}. \tag{6}$$

Cada par de ecuaciones es una «mariposa». Aplicado de forma recursiva (con N potencia de 2), el costo cumple $T(N) = 2T(N/2) + \Theta(N)$, es decir:

$$\begin{aligned} &T_{\mathrm{FFT}}(N) = \Theta(N \log_2 N), \\ &\tfrac{N}{2}\log_2 N = 5.120 \text{ mariposas para } N = 1024, \qquad \tfrac{N^2}{(N/2)\log_2 N} \approx 205. \end{aligned} \tag{7}$$

La implementación es iterativa y en el lugar: primero reordena las muestras con la permutación de inversión de bits y luego ejecuta $\log_2 N$ etapas de mariposas, con los factores W_N^k precalculados una sola vez. La relación teórica de operaciones es ≈ 205; la aceleración medida en la práctica fue de 88× para $N = 1024$ (Tabla 1).

Tabla 1. Tiempo medido de una transformada de N puntos (Node.js, entrada compleja aleatoria)

N	FFT (ms)	DFT (ms)	Aceleración	Error máx.
64	0,0019	0,017	9×	$7{,}1\times10^{-15}$
128	0,0036	0,048	13×	$1{,}4\times10^{-14}$
256	0,0066	0,188	28×	$2{,}1\times10^{-14}$
512	0,0127	0,629	49×	$5{,}0\times10^{-14}$
1024	0,0287	2,521	88×	$1{,}1\times10^{-13}$
2048	0,0670	10,462	156×	$2{,}9\times10^{-13}$
4096	0,1534	39,535	258×	$6{,}2\times10^{-13}$

Código: `lib/huella.js` → `plan`, `fft`. Mediciones: `demo/scripts/experimentos.js` → `fftVsDft`.

4. Ventana de Hann

Una señal real se analiza en bloques finitos. Cortar la señal de golpe equivale a multiplicarla por una ventana rectangular y «mancha» la energía de cada nota hacia bins vecinos (fuga espectral). Para atenuar esos bordes cada bloque se multiplica por la ventana de Hann:

$$w[n] = \frac{1}{2} - \frac{1}{2}\cos\left(\frac{2\pi n}{N}\right), \qquad 0 \le n < N. \tag{8}$$

Vale 0 en los extremos y 1 en el centro ($n = N/2$). Propiedades conocidas de esta ventana: el lóbulo principal ocupa ±2 bins, el primer lóbulo lateral está ≈ 31 dB por debajo y su ancho de banda equivalente de ruido es 1,5

bins. Esas son las que hacen que dos notas próximas se vean como crestas separadas en el espectrograma.

Código: `lib/huella.js` → `hann` (verificada en las pruebas: $w[0]=0$ y $w[N/2]=1$).

5. Transformada de tiempo corto y espectrograma

La música cambia en el tiempo, por lo que se aplica la FFT a bloques que se van desplazando. Con ventana w, tamaño N y salto H (en muestras), la transformada de tiempo corto de la trama m es

$$X[m,k]=\sum_{n=0}^{N-1} w[n]\,x[mH+n]\,e^{-j2\pi kn/N}, \qquad M=\left\lfloor \frac{L-N}{H} \right\rfloor + 1, \tag{9}$$

donde L es la longitud de la señal y M el número de tramas. El espectrograma es la potencia en decibelios (se suma 10^{-12} para evitar $\log 0$ en el silencio digital):

$$S[m,k]=10\log_{10}\!\left(|X[m,k]|^2+10^{-12}\right)\ \text{dB}, \qquad t_m=\frac{mH}{f_s}\ \text{s}. \tag{10}$$

Con $N=1024$ y $H=256$ muestras a $f_s=8.000$ Hz: cada ventana dura **128 ms**, hay una trama nueva cada **32 ms** (31,25 tramas por segundo, solape del 75 %) y la resolución es 7,8125 Hz por bin. Solo se usan los primeros 256 bins (0–2000 Hz), donde está la información musical útil. Existe un compromiso inevitable: una ventana más larga mejora Δf pero empeora la resolución temporal.

Código: `lib/huella.js` → `espectrograma`.

6. Detección de picos (constelación)

La huella no guarda todo el espectrograma, solo sus máximos locales: son los puntos más fuertes y, por eso, los que mejor resisten el ruido. Un punto (m,k) es un pico si cumple a la vez tres condiciones:

$$\begin{aligned} &S[m,k]=\max_{\substack{|m'-m|\le \Delta_t\\ |k'-k|\le \Delta_f}} S[m',k'],\\ &S[m,k] > S_{\max}-\gamma,\\ &k \ge k_{\min}, \end{aligned} \tag{11}$$

con $\Delta_t=8$ tramas, $\Delta_f=10$ bins, margen $\gamma=12$ dB respecto del máximo global $S_{\max}$ y $k_{\min}=4$ (se ignoran las frecuencias menores que 31,3 Hz). El máximo en una vecindad rectangular se calcula de forma **separable** (primero a lo largo de frecuencia y luego del tiempo), lo que reduce el costo de $O(TK\,\Delta_t\Delta_f)$ a $O(TK(\Delta_t+\Delta_f))$:

$$\begin{aligned} M_1[m,k]&=\max_{|k'-k|\le\Delta_f} S[m,k'],\\ M_2[m,k]&=\max_{|m'-m|\le\Delta_t} M_1[m',k],\\ (m,k)\ \text{es pico} &\iff S[m,k]=M_2[m,k]. \end{aligned} \tag{12}$$

El criterio de máximo local reparte los picos por todo el plano tiempo-frecuencia y es insensible a la ganancia global, porque compara decibelios entre sí. El margen γ descarta lo que queda bajo el ruido.

Código: `lib/huella.js` → `picos`.

7. Pares de picos y hash de 24 bits

Un pico aislado es poco selectivo (solo 9 bits de frecuencia). Por eso cada pico *ancla* (m_1, k_1) se empareja con los siguientes $F = 10$ picos *objetivo* (m_2, k_2) situados hasta 63 tramas después, y cada par produce un hash con ambas frecuencias y su separación temporal:

$$h = \big(k_1 \cdot 512 + k_2\big) \cdot 64 + \Delta m, \qquad \Delta m = m_2 - m_1 \in [1,\, 63], \qquad h \in [0,\, 2^{24}). \tag{13}$$

Reparto de bits: k_1 (9 bits) + k_2 (9 bits) + Δm (6 bits) = 24 bits, es decir, 16.777.216 valores posibles. La clave del método es que h depende de Δm y no del instante absoluto del par, así que el mismo trozo de canción produce los mismos hashes aunque el fragmento empiece en cualquier punto. Con P picos se generan como máximo $F \cdot P$ hashes.

Código: `lib/huella.js` → `hashes`, `huella`.

8. Índice invertido y votación

Al registrar una canción, cada hash h se guarda con el identificador de la canción s y su instante m_{bd}, empaquetados en un solo entero (tiempo hasta 2^{21} tramas):

$$e = s \cdot 2^{21} + m_{\text{bd}}, \quad s = \left\lfloor \tfrac{e}{2^{21}} \right\rfloor, \quad m_{\text{bd}} = e \bmod 2^{21}. \tag{14}$$

El índice es un mapa $h \mapsto \{e\}$, de modo que buscar un hash cuesta $O(1)$. En la consulta, cada hash (h, m_{q}) recupera todas las entradas con el mismo h y vota por el desplazamiento entre el instante de la base de datos y el de la consulta:

$$\delta = m_{\text{bd}} - m_{\text{q}}. \tag{15}$$

Si el fragmento pertenece a la canción s, muchos pares coinciden con el *mismo* δ (la posición del fragmento dentro de la canción), mientras que las coincidencias por azar se reparten entre desplazamientos distintos. Sea $c_s(\delta)$ el número de votos de s con desplazamiento δ. El puntaje de una canción, la decisión y la posición estimada son:

$$\begin{aligned} \text{puntaje}_s &= \max_{\delta} \big[c_s(\delta-1) + c_s(\delta) + c_s(\delta+1)\big], \\ \hat{s} &= \arg\max_{s}\ \text{puntaje}_s, \qquad \hat{t} = \frac{\hat{\delta}\, H}{f_s}\ \text{s}. \end{aligned} \tag{16}$$

Se suman tres desplazamientos consecutivos porque el fragmento empieza en una muestra arbitraria y los instantes se cuantizan en pasos de 32 ms, lo que desplaza algunos picos una trama. La canción $\hat{s}$ se acepta solo si $\text{puntaje}_{\hat{s}} \geq \tau$, con umbral $\tau = 12$ **votos**; si no, la respuesta es «sin coincidencia fiable». La posición $\hat{t}$ puede ser ambigua si la pieza repite frases.

Código: `lib/huella.js` → `Indice.agregar`, `votar`, `puntajes`, `buscar`; `server.js` → `POST /api/reconocer`.

9. Ruido y relación señal-ruido

Para evaluar la robustez se le suma al fragmento ruido blanco gaussiano. La relación señal-ruido (SNR) compara la potencia de la señal P_x con la varianza del ruido σ^2:

$$\begin{aligned} P_x &= \frac{1}{L}\sum_{i=0}^{L-1} x[i]^2, \qquad \text{SNR}_{\text{dB}} = 10\log_{10}\frac{P_x}{\sigma^2}, \\ \sigma^2 &= \frac{P_x}{10^{\text{SNR}_{\text{dB}}/10}}, \qquad y[i] = x[i] + \sigma z_i. \end{aligned} \tag{17}$$

Las variables gaussianas $z_i \sim \mathcal{N}(0,1)$ se generan con la transformada de Box–Muller a partir de dos números uniformes $u_1, u_2 \in (0,1)$:

$$z_1 = \sqrt{-2\ln u_1}\,\cos(2\pi u_2), \qquad z_2 = \sqrt{-2\ln u_1}\,\sin(2\pi u_2). \tag{18}$$

0 dB significa ruido tan fuerte como la señal; −10 dB, ruido diez veces más potente. Los experimentos usan un generador pseudoaleatorio con semilla fija (*mulberry32*) para que los resultados sean reproducibles.

Código: `lib/sintesis.js` → `agregarRuido`, `mulberry32`; `public/app.js` → `agregarRuido` (deslizador de la demo).

10. Colisiones de hash

Si los U hashes distintos del índice estuvieran repartidos de manera uniforme en 2^{24} valores, un hash cualquiera de la consulta coincidiría por azar con probabilidad

$$p \approx \frac{U}{2^{24}} = 0{,}042\ \% \quad (U = 7.045 \text{ con } 12 \text{ canciones}). \tag{19}$$

La medición real fue 2,0 % (48 veces mayor), porque la música sintética (y la real) no usa el espacio de manera uniforme: las notas ocupan pocas frecuencias y los ritmos pocos intervalos. Por eso la decisión **no** se basa en que «haya coincidencias», sino en que se **concentren en un desplazamiento** (ecuación (16)).

Código: `demo/scripts/experimentos.js` → `costos`.

11. Síntesis de las canciones de prueba

Para no usar material con derechos de autor, la base se genera por código. Cada canción se compone de notas; la frecuencia de la nota MIDI n en afinación temperada, con un desafinado propio de la pieza $c \in [-50, 50]$ centésimas, es

$$f_0(n, c) = 440 \cdot 2^{\frac{n-69}{12} + \frac{c}{1200}}\ \text{Hz}. \tag{20}$$

Cada nota de duración d es una suma de armónicos, multiplicada por una envolvente de ataque lineal de 10 ms, caída exponencial y liberación lineal de 30 ms:

$$\begin{aligned} x(t) &= a\,A(t) \sum_{h=1}^{4} w_h \sin(2\pi h f_0 t), \\ A(t) &= \min\!\left(1, \tfrac{t}{0{,}01}\right) \cdot \max\!\left(0, \min\!\left(1, \tfrac{d-t}{0{,}03}\right)\right) \cdot e^{-t/\tau}, \\ \tau &= \frac{\min(d,\ 0{,}7)}{2{,}5}. \end{aligned} \tag{21}$$

Los pesos de los armónicos son $w = [1,\ 0{,}25{+}0{,}4u_1,\ 0{,}08{+}0{,}3u_2,\ 0{,}03{+}0{,}12u_3]$ con u_i uniformes (cada pieza tiene su timbre), y un armónico solo se incluye si $h f_0 < f_s/2 - 200$ Hz, para evitar aliasing. Las amplitudes son $a = 0{,}35$ (melodía), $0{,}30$ (bajo) y $0{,}12$ (arpegio). El tempo es $\text{bpm} \in [80, 139]$, de modo que un pulso dura $60/\text{bpm}$ segundos. La melodía son frases de 8 pulsos sobre una escala (mayor, menor, pentatónica o dórica) que se repiten en el orden A–B–A–C. Finalmente la señal se normaliza para que su pico sea 0,8:

$$x \leftarrow 0{,}8\,\frac{x}{\max_i |x[i]|}. \tag{22}$$

Código: `lib/sintesis.js` → `midiAHz`, `cancion`.

12. Visualización 2D y 3D

Para dibujar el espectrograma, la potencia se normaliza a un rango de 70 dB bajo el máximo y se usa como color (ambas vistas) y como altura (vista 3D):

$$v = \operatorname{clamp}\left(\frac{S[m,k] - (S_{\max} - 70)}{70},\, 0,\, 1\right), \qquad y = v^{1{,}9} \cdot y_{\max} \cdot e, \tag{23}$$

donde e es el control de «Altura». El exponente 1,9 aplana el ruido de fondo (valores bajos de v) y realza las notas (valores cercanos a 1). El color es una interpolación lineal por tramos entre cinco paradas (negro $\to$ azul $\to$ magenta $\to$ naranja $\to$ amarillo en $v = 0;\ 0{,}35;\ 0{,}6;\ 0{,}82;\ 1$). La malla 3D toma, en cada celda (2 bins de frecuencia × varias tramas), el **máximo** de S para no perder picos al reducir la resolución, y sus ejes son tiempo ($t = mH/f_s$) y frecuencia ($f = kf_s/N$).

Código: `lib/colores.js` → `colorMapa`; `public/spectro3d.js`; `public/app.js` → `dibujar2D`.

13. Métricas de evaluación

Con n consultas de canciones de la base y una canción verdadera s_i para cada una, se reportan:

$$\begin{aligned} \text{top-1} &= \frac{1}{n}\sum_{i=1}^{n} 1[\hat{s}_i = s_i], \\ \text{aceptadas} &= \frac{1}{n}\sum_{i=1}^{n} 1[\hat{s}_i = s_i \ \wedge\ \text{puntaje} \geq \tau], \\ \text{FPR} &= \frac{\#\{\text{canciones ajenas aceptadas}\}}{\#\{\text{consultas ajenas}\}}. \end{aligned} \tag{24}$$

«Top-1» mide si la canción correcta fue la más votada; «aceptadas» exige además superar el umbral; y la tasa de falsos positivos (FPR) se mide consultando con canciones que *no* están en la base. La FFT se evalúa por su aceleración respecto de la DFT, $t_{\text{DFT}}/t_{\text{FFT}}$, y por el error numérico máximo entre ambas, $\max_k |X_{\text{FFT}}[k] - X_{\text{DFT}}[k]|$ ($\approx 10^{-13}$, del orden del redondeo en doble precisión).

Código: `demo/scripts/experimentos.js`. Resultados completos: `informe/resultados.json` y el informe IEEE.

14. Tabla de parámetros y mapa del código

Tabla 2. Parámetros de la huella acústica (valores reales de `lib/huella.js`)

Símbolo	Significado	Valor
f_s	Frecuencia de muestreo	8.000 Hz
N	Tamaño de ventana y de FFT	1024 muestras (128 ms)
H	Salto entre ventanas	256 muestras (32 ms)
$\Delta_t,\ \Delta_f$	Vecindad de un pico	±8 tramas, ±10 bins
γ	Margen bajo el máximo global	12 dB
$k_{\min}$	Primer bin considerado	4 (31,3 Hz)
F	Pares por ancla	10
$\Delta m_{\max}$	Separación máxima ancla–objetivo	63 tramas
τ	Umbral de decisión	12 votos

Tabla 3. Dónde se implementa cada modelo

Modelo	Archivo	Función
DFT, FFT, ventana de Hann	`lib/huella.js`	`dft`, `fft`, `hann`
Espectrograma	`lib/huella.js`	`espectrograma`
Picos, pares y hashes	`lib/huella.js`	`picos`, `hashes`, `huella`
Índice y votación	`lib/huella.js`	`Indice.agregar / buscar`
Síntesis, ruido, WAV	`lib/sintesis.js`	`cancion`, `agregarRuido`, `leerWav`
Mapa de color	`lib/colores.js`	`colorMapa`
Vista 3D y 2D	`public/spectro3d.js`, `public/app.js`	`crearEspectro3D`, `dibujar2D`
Servidor de búsqueda	`server.js`	`POST /api/reconocer`
Experimentos y pruebas	`scripts/experimentos.js`, `test/huella.test.js`	—